

Projeto prático Prog2 – Pacman em C usando Raylib.

Nome: Gabriel Maia Marinho DRE: 120124590

Resolvi fazer esse trabalho sozinho e descobri que realmente era bastante coisa, porém consegui aplicar tudo que foi pedido, não consegui aplicar novas modificações porém gostei do desafio.

Tive a ideia de usando o GITHUB e como uso Windows tive que baixar o GCC para compilar no meu notebook, além disso pra rodar o Raylib da forma pedida tive que configurar o terminal UCTR64 dentro do Vscode pelo Json de analise.

Após fazer isso pedi pra duas IA's fazerem os códigos do jogo para eu ter um exemplo de base e organização. A partir disso comecei a minha organização:

Criei a pasta do jogo principal e três diretórios para colocar as funções e strutus; Include para as estruturas, Src para as funções, um Maps para os mapas de exemplo e um Makefile para conseguir compilar. Cada parte do código esta bem comentada e as funções e lógicas também, a única coisa que tive q copiar foi o MakeFile pois não estou acostumado com esse tipo de arquivo e como ele funcionava, mas tive sim problema com ele.

Eu comitei tudo no meu GitHub <https://github.com/Gmmarinho07/pacman-prog2-2025> e nele pode ver o período e o que fiz em cada commit.

Demorei cerca de um mês para avaliar os códigos e começar o meu, o que tomou um tempo grande.

Implementei o que foi pedido da melhor forma que conseguir:

- **Mapa e Renderização:** O mapa de 20 por 40 blocos é carregado dinamicamente via arquivo texto. Todos os elementos (Parede, Pellet, Power Pellet, Pac-Man, Fantasma e Portal) são renderizados utilizando formas geométricas simples e as cores especificadas (Ex: Power Pellet verde, Portal rosa).

- **Movimentação e Física:** O movimento do Pac-Man e dos Fantasmas é discreto (1 bloco por passo) e responsivo, utilizando `IsKeyDown` para capturar a direção imediatamente. Paredes são tratadas como obstáculos intransponíveis, e os portais teletransportam os personagens instantaneamente.
- **Lógica do Jogo:**
 - **Colisão/Pontuação:** O Pac-Man coleta pellets (+10) e Power Pellets (+50). Colisão com Fantasma não vulnerável resulta em perda de vida e -200 pontos (mínimo 0). No modo FRIGHTENED, o Fantasma é comido (+100 pontos) e reseta para a base.
 - **Movimentação dos Fantasmas:** Implementação da **inteligência mínima**: Fantasmas seguem em linha reta até uma interseção e escolhem uma direção válida aleatória, evitando a reversão imediata¹⁰. A velocidade é ajustada (4 blocos/s normal, 3 blocos/s no modo vulnerável).
 - **Heads-up Display (HUD):** Exibição do placar (6 dígitos), vidas, nível e pellets restantes.
 - **Menu de Opções:** O menu é acessível via TAB, permitindo *Novo Jogo*, *Voltar ao Jogo*, *Salvar*, *Carregar* e *Sai*.

3. Decisões de Projeto e Estrutura de Dados ¹⁴

- **Alocação Dinâmica:** O mapa do jogo (Map->tiles) é alocado dinamicamente como um vetor unidimensional para facilitar a leitura binária e atender ao requisito de alocação flexível¹⁵.
- **Estruturas (Structs):** A posição dos elementos é gerenciada por struct `Position {int row; int col; Direction direction;}`. O Pac-Man, Fantasmas e o estado do jogo (GameState) são estruturas que encapsulam suas propriedades e estado.
- **Sistema de Tick (Velocidade):** Para garantir o deslocamento discreto e a velocidade em blocos/s, utilizamos o sistema de `moveTimer` e `moveDelay`. O tempo do frame (dt) é subtraído de `moveTimer`, e a

entidade se move apenas quando `moveTimer <= 0`, garantindo a precisão temporal.

- **Salvar/Carregar:** Implementação obrigatória utilizando **arquivo binário** (`pacman_save.bin`). Os dados críticos são serializados (copiados) para uma struct `GameSaveData` estática antes de serem escritos (função `SaveGame`) e restaurados (função `LoadGame`)¹⁸.